Edge AI-Based FPGA Accelerator for Spectrum Classification and Anomaly Detection

Author 1, Author 2, and Author 3

***Abstract*—Radio Frequency (RF) spectrum monitoring is critical for modern electromagnetic signal environments, where non-cooperative signals, including jamming pulses, spoofing transmissions, and wide-band interference, must be classified and countered within microseconds of first reception. This work presents a fully synthesizable field-programmable gate array (FPGA) accelerator for real-time RF anomaly detection at the sensor edge, eliminating dependence on cloud connectivity or external memory. The system implements an eight-layer one-dimensional Residual Convolutional Neural Network (ResNet-1D). The trained model achieves 83.23% test accuracy in 32-bit floating-point (FP32) precision and 83.18% after 8-bit integer (INT8) symmetric post-training quantization, representing a degradation of only 0.05% points. Hardware deployment is enabled through Batch Normalization (BatchNorm) folding and symmetric per-tensor INT8 quantization, compressing all weights to 28.6 KB within the Block Random Access Memory (BRAM) budget of a Xilinx Zynq-7000 System-on-Chip (SoC). The synthesized Register-Transfer Level (RTL) datapath, governed by an eight-state Moore Finite State Machine (FSM) clocked at 72.2 MHz, achieves an inference latency of 0.78 μs, which is over four orders of magnitude faster than Graphics Processing Unit (GPU) baselines. The accelerator consumes ultra-low on-chip power of 0.671 W, yielding an energy efficiency of 0.38μJ per inference, which makes the accelerator suitable to be deployed in a battery-powered real-time spectrum monitoring environment.**

***Index Terms*—Edge AI, FPGA accelerator, RF anomaly detection, ResNet-1D, Spectrum monitoring.**

I. Introduction

Modern electromagnetic spectrum monitoring systems must be designed to respond to the electromagnetic environment in near real-time. A jammer can suppress communications by barrage or spot-frequency jamming, a navigation spoofer can manipulate receiver state without the receiver's knowledge, and wideband interference from co-channel transmitters can degrade radar performance. The system must detect, classify, and respond to the anomalous waveform within microseconds of its arrival. The Field-Programmable Gate Arrays (FPGAs) present a compelling hardware platform. They synthesize completely pipelined custom datapaths for Convolutional Neural Networks (CNNs) without any of the Instruction Fetch, Decode or cache-miss latency as experienced by a Central Processing Units (CPUs)/GPU. This allows FPGA-synthesized CNN inference accelerators to enjoy huge spatial parallelism across filter taps, achieve theoretical Multiply-Accumulate (MAC) throughput, and realize deterministic latency for spectrum-monitoring applications. However, the application of deep neural networks in FPGA logic imposes stringent engineering constraints on floating-point operations, which are prohibitively expensive in terms of large look-up tables (LUTs) and digital signal processing (DSP) slice usage. Per-channel running statistics in batch normalization (BN) operations necessitate hardware registers for each channel, and training in the presence of vanishing gradients affects both deep plain CNNs' training consistency and accuracy.

In recent years, an end-to-end learning framework for automatic modulation classification based on raw in-phase (I) and quadrature (Q) samples has become the mainstream of automatic modulation classification (AMC) research [1], [2]. The RadioML 2018.01A dataset [3] subsequently provided 24-class I/Q data at 26 Signal-to-Noise Ratio (SNR) levels and became the community standard benchmark. The authors in [4] demonstrated the efficiency of an attention-enhanced, transformer-based deep learning architecture. The authors in [5] have developed a systematic description of robust deep learning for physical layer communications. Recent literature on FPGA-based AI accelerators [6] categorizes optimizations at the computational, memory, and system-level, emphasizing hand-designed RTL and extreme quantization as key for performance in resource-limited systems. Also, the authors in [7] proved that the models built using the attention mechanism obtain much better performance on wireless modulation classification. Existing FPGA deep learning RF classifiers, such as the DeepRadio platform [8], classify signals received through the RF front end to different modulation types in real time with low power, targeting real-time RF classification applications. The RF System-on-Chip (RFSoC) work in [9] uses VGG10 and ResNet33 models on the RadioML 2018.01A dataset with FPGA deployment. While prior work in [10] demonstrated an FPGA-accelerated anomaly detector utilizing an unsupervised autoencoder for generic, out-of-distribution flagging, our work distinctly frames the problem around a supervised ResNet-1D architecture mapping directly to a robust, operational signal taxonomy. In [11], the authors achieved approximately 95% classification accuracy using a ResNet with six residual stacks on the full 24-class RadioML dataset. Tools like hls4ml already implement optimizers that fuse batch normalizations following affine projection layers into a single node with equivalent fused weights [12], and ResNet INT8 inference on Xilinx devices is established practice. Recently, [13] surveyed deep transfer learning approaches for AMC, highlighting emerging architectures that improve generalization across diverse channel conditions. The authors in [14] demonstrated an FPGA-based real-time signal measurement system using deep learning accelerators.

The limitations motivate an end-to-end hardware-accelerated pipeline implemented on an FPGA in this

work; the major contributions are summarized as follows.

- We introduce a novel four-class operational signal taxonomy that maps the 24 RadioML 2018.01A modulation classes onto operational signal categories (Normal, Jammer, Spoofer, Interference), reframing fine-grained modulation recognition as an operationally meaningful signal-classification problem.
- We design an eight-layer ResNet-1D architecture, optimized for FPGA deployment through symmetric post-training INT8 quantization and algebraic BatchNorm folding, compressing the full model to under 30 KB of BRAM. That INT8 post-training quantization preserves 83.18% test accuracy with only 0.05 percentage-point degradation from the FP32 baseline while compressing all weights to 28.6 KB within on-chip BRAM.
- We implement a fully synthesizable Verilog RTL datapath with zero HLS tooling dependencies, achieving an inference latency of 0.78 μs at 72.2MHz, over four orders of magnitude faster than GPU baselines, with a total low on-chip power of only 0.671W and an energy cost of 0.38 μJ per inference, utilizing just 18.82% of LUTs, 5.13% of flip-flops, and 10.71% of BRAM on the Xilinx Zynq-7000 (xc7z020) ZedBoard.

The remainder of this paper is organized as follows. Section II presents the proposed architecture; Section III gives the details of the FPGA implementation, Section IV gives the results, and Section V concludes the work.

II. Proposed Architecture

The RadioML 2018.01A dataset [3], provides 2,555,904 labeled I/Q signal records across 24 modulation types at 26 SNR levels from -20 dB to $+30$ dB in 2 dB steps. Each record is a floating-point array of shape $(1024, 2)$ representing 1,024 complex-valued time samples with I/Q baseband components. The dataset models realistic channel impairments including additive white Gaussian noise (AWGN), multipath Rayleigh fading, carrier frequency offset, timing recovery errors, and hardware non-linearities such as IQ imbalance and phase noise. Standard modulation recognition assigns one label per modulation type, yielding a 24-class problem with fine-grained distinctions, e.g., between 16 Quadrature Amplitude Modulation (QAM) and 32 QAM. For real-time spectrum monitoring applications, this granularity is operationally unnecessary and harmful: it increases classifier complexity while the system only needs to know whether a signal is benign or what operational signal category it represents. This work instead introduces a four-class operational signal taxonomy that consolidates the 24 modulations into operationally aligned groups based on spectral characteristics, bandwidth occupancy, and operational signal profiles.

A plain stacked 1D CNN applies convolutional layers sequentially with no bypass paths. During backpropagation, the gradient of the loss with respect to early-layer parameters is computed as a product of Jacobian matrices across every intermediate layer. ResNet-1D maps a two-channel 128-point I/Q input to four anomaly class logits through eight computational stages. The architecture employs progressively increasing channel widths ($2 \to 16 \to 32 \to 64$) combined with stride-2 downsampling at two points to compress the temporal dimension from 128 to 32 timesteps before global average pooling. Table I provides a complete layer-wise breakdown. The 1D convolution output for output channel index c_o at time position t is expressed as

TABLE I
ResNet-1D Layer-wise Architecture

Layer index	Type	In (C×L)	Out (C×L)	k, s	Parameters
1	Convolution+BN+ReLU	2×128	16×128	7, 1	240
2	Convolution+BN+ReLU	16×128	32×64	3, 2	1,568
3–4	ResBlk 1 (id)	32×64	32×64	3, 1	9,280
5–7	ResBlk 2 (proj)	32×64	64×32	3, 2	17,664
8	GAP	64×32	64×1	—	0
9	FC + Argmax	64	4	—	260
			Total Trainable Parameters		**29,252**

ResBlk: Residual Block; id: identity shortcut; proj: projection shortcut; GAP: Global Average Pooling; FC: Fully Connected layer; ReLU: Rectified Linear Unit; C: channels; L: temporal length; s: stride; k: kernel size.

$$y[c_o, t] = \sum_{c_i=0}^{C_{in}-1} \sum_{k=0}^{K-1} W[c_o, c_i, k] * x[c_i, t \cdot s + k] + b[c_o], \quad (1)$$

where $*$ denotes the linear convolution operation, c_i is the input channel index, K is the total kernel size, x is the input tensor, W is the weight tensor, $[\cdot, \cdot, \cdot]$ denotes tensor indexing, C_{in} is the number of input channels, and $b[c_o]$ is the bias. Batch Normalization normalizes each channel, and the normalized output for each channel is expressed as

$$\hat{x}[c] = \gamma[c] \frac{x[c] - \mu[c]}{\sigma[c]} + \beta[c], \quad (2)$$

where $x[c]$ is the input for channel c, $\mu[c]$ and $\sigma[c]$ are the channel's running mean and standard deviation, and $\gamma[c]$ and $\beta[c]$ are the learned scale and shift parameters, respectively. ReLU activation applies element-wise, $\mathrm{ReLU}(x) = \max(0, x)$, where x is the input value and $\max$ returns the maximum of its arguments. The identity skip connection computes $y = F(x) + x$, when channel dimensions match; when they do not, a 1×1 projection convolution adjusts the identity as $y = F(x) + W_p * x$, where x is the input tensor, y is the output tensor, $F(x)$ is the residual mapping function, and W_p is the 1×1 projection weight matrix. Global Average Pooling reduces each channel to a scalar. The pooled output scalar for channel c is given as $z[c] = \frac{1}{T} \sum_{t=0}^{T-1} x[c, t]$, where T is the temporal sequence length, t is the time step index, and $x[c, t]$ is the input value at channel c and time step t. Since $T = 32$, the hardware division is replaced by a signed arithmetic right shift of 5 positions, eliminating all division circuitry from the pooling datapath. The argmax classification selects $\hat{c} = \arg\max_{j \in \{0,1,2,3\}} l[j]$, and an anomaly flag is asserted whenever $\hat{c} \neq 0$, where $\hat{c}$ is the predicted class index,

$\arg\max$ is the operation returning the argument that gives the maximum value, j is the class index iterating over the four classes, and $l[j]$ is the logit score for class j.

III. INT8 Quantization and FPGA Implementation

This work employs symmetric per-tensor post-training quantization (PTQ) [18]. Asymmetric quantization requires a zero-point subtraction in every MAC operation, expanding to four terms per multiply-accumulate and doubling the effective multiply count in hardware. Symmetric quantization ($z = 0$) keeps the MAC as a single signed multiply with no offset accumulation, saving significant LUT and DSP resources. For a floating-point weight tensor W, the quantization scale factor s_W maps the maximum absolute weight value to the INT8 ceiling of 127, given as $s_W = \frac{\max(|W|)}{2^{b-1}-1}$, where b is the quantization bit-width, max denotes the maximum function, and $|W|$ denotes the element-wise absolute value of the weight tensor W. Each weight element is quantized by rounding to the nearest integer and clamping to $[-128, 127]$. The resulting quantized weight tensor is expressed as

$$W_q = \text{clamp}\left(\text{round}\left(\frac{W}{s_W}\right), -128, 127\right), \tag{3}$$

where round (.) is the round-to-nearest-integer operation, and clamp$(v, \min, \max)$ restricts the value v to the range $[\min, \max]$. Prior to quantization, every BatchNorm layer is algebraically absorbed into its preceding convolution. The folded weight is computed per output channel c_o as

$$\tilde{W}[c_o, c_i, k] = \frac{\gamma[c_o]}{\sigma[c_o]} W[c_o, c_i, k], \tag{4}$$

where c_i denotes the input channel, k denotes the kernel index, $W[c_o, c_i, k]$ is the original convolution weight, and $\gamma[c_o]$ and $\sigma[c_o]$ are the BatchNorm scale and standard deviation for output channel c_o. The folded bias absorbs both the original convolution bias and the complete BN affine transformation. The folded bias for output channel c_o is expressed as

$$\tilde{b}[c_o] = \frac{\gamma[c_o] \cdot (b[c_o] - \mu[c_o])}{\sigma[c_o]} + \beta[c_o], \tag{5}$$

where $b[c_o]$ is the original convolution bias, and $\mu[c_o]$ and $\beta[c_o]$ are the BatchNorm running mean and shift parameter for output channel c_o. Applying this across all six Conv–BN pairs in ResNet-1D removes every running-mean accumulator, running-variance accumulator, and per-channel divider from the hardware, reducing RTL complexity significantly.

Each MAC operation multiplies a signed INT8 weight by a signed INT8 input activation, producing a signed 16-bit product. Products are accumulated across all $C_{in}K$ terms into a signed 24-bit register. The maximum accumulation magnitude for the worst-case layer ($C_{in}K = 32 \times 3 = 96$ terms) is $96 \times 127^2 = 1,548,384$, requiring only 21 bits plus sign—well within the 24-bit accumulator capacity of 8,388,608. After the full dot product and bias addition, the result is saturated back to INT8: $\text{out}_8 = \text{clamp}(\text{acc}_{24} + b_q, -128, 127)$, where out_8 is the saturated 8-bit output, acc_{24} is the 24-bit accumulated dot product, and b_q is the quantized folded bias. This model uses PTQ rather than quantization-aware training (QAT) because QAT incorporates simulated quantization noise during the training loop, allowing the network to adapt its weight distribution to the quantization grid.

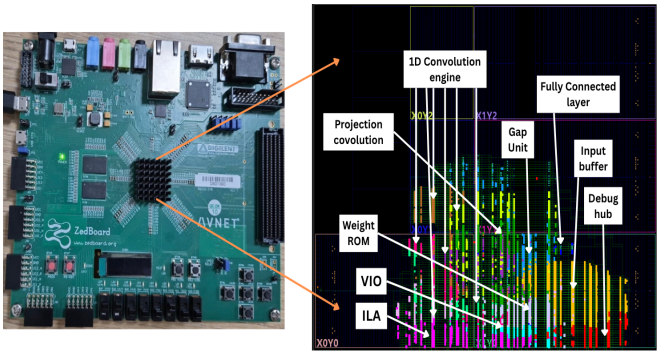


Fig. 1. Physical floorplan and post-implementation resource mapping of the proposed adaptive-precision accelerator on the xc7z020 FPGA fabric.

Fig. 1 illustrates the physical deployment on the ZedBoard platform alongside its post-implementation Vivado floorplan which highlights the physical location of various on-chip units like 1D convolution engines, Weight ROM, and built-in debugging IPs on chip and verifies the extremely small space occupied by the INT8-quantized design that efficiently maps ResNet-1D to the Zynq-7000 logic fabric without encountering any routing overflow. A top-level module implements the eight-state Moore FSM shown in Fig. 2, clocked at 72.2 MHz and targeting Xilinx 7-Series FPGA platforms. In state **IDLE**, wait for start pulse, which indicates the start of the process, clear busy flag, which indicates if the FSM is doing some operation in a state, de-assert done, which is used to signal the process is over after reaching the final state, res_valid (result valid) are all set to low. After starting, state **LOAD** stores 128 I/Q data pairs whenever iq_valid is high. State **CONV_IN** starts convolution using the first two layers, waits until conv_in_done, which signals the end of the convolution operation, and becomes true. In states **RES_BLK1** and **RES_BLK2**, we implement respectively residual blocks with identity skip and 1×1 projection skip, waiting until res_blk1_done and res_blk2_done, which indicate the completion of the above-mentioned operations, become true. In state **GAP**, global average pooling is executed, and gap_done is asserted when this execution is completed. In state **FC_OUT**, 64×4 matrix multiplication is carried out, then the argument of the maximum function is run, which finds the index of the largest value in a list. Finally, anomaly_flag is set to high if the prediction is not class 0. In state **DONE**, capture class_id, raise done and res_valid signal to report completion, save total inference cycle, then return to **IDLE**.

The RTL is a fully custom, hand-written Verilog implementation with zero HLS tooling dependencies. The architecture includes a parameterized INT8 MAC engine, residual blocks with identity and 1×1 projection convolutions, arithmetic-

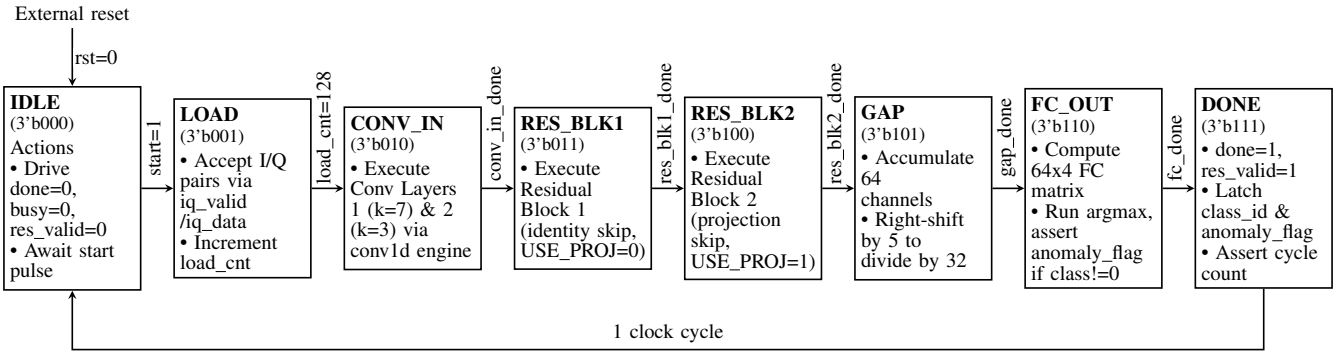


Fig. 2. Eight-state Moore FSM controlling inference datapath. Each state maps to a dedicated hardware module, ensuring deterministic latency and zero resource conflicts.

shift-based GAP, and a combinational argmax with binary anomaly flagging. Every module was derived directly from the governing equations and verified cycle-accurately against Python golden references. This contrasts with HLS-based approaches that sacrifice RTL transparency and fine-grained resource control.

The MAC engine traverses output channels, time positions, input channels, and kernel taps in a four-level nested FSM, fully utilizing all INT8 multipliers and maximizing throughput from each available DSP slice. Three key resource optimizations distinguish this work from baseline FPGA CNN implementations: BN folding eliminates all six BN layers from the RTL, removing running-mean accumulators, running-variance accumulators, and per-channel dividers entirely; GAP replaces division by $T = 32$ with a 5-bit arithmetic right shift, consuming zero multiplier cells in the pooling datapath. INT8 symmetric quantization compresses weight storage to 28.6 KB, fitting in a single 36 KB BRAM block on the Zynq-7000, avoiding multi-BRAM arbitration overhead.

The complete co-design stack spans five stages: (1) Python preprocessing generating INT8 test vectors; (2) PyTorch BN folding and quantization producing hex files; (3) weight export scripts merging into a 32,768-entry Read-Only Memory (ROM); (4) cycle-accurate simulation targeting the generated golden vectors; and (5) Vivado RTL synthesis with deep hardware debug integration for on-device ZedBoard validation. Specifically, an Integrated Logic Analyzer (ILA) core triggers on the `start` pulse to capture cycle-level waveform data on the physical FPGA. A Virtual Input/Output (VIO) core provides real-time software dashboard control over inputs like `iq_data` and probes state outputs such as `anomaly_flag` and `result_valid`.

IV. Results

This section evaluates the proposed system along two axes: classification performance of the ResNet-1D model under the four-class operational signal taxonomy, and the resource, timing, and power characteristics of the synthesized RTL accelerator on the Xilinx Zynq-7000 ZedBoard platform. The ResNet-1D model was trained using PyTorch with the Adam optimizer [19] at an initial learning rate of 1×10^{-3} and weight decay 1×10^{-4}. A cosine annealing learning rate schedule reduced the rate to $\eta_{\min} = 1 \times 10^{-6}$ over a 60-epoch budget. Early stopping with patience 10 terminated training at epoch 50. Cross-entropy loss was used with a mini-batch size of 256.

TABLE II
Per-Class Performance on Test Set

Class	Precision (%)	Recall (%)	F1 (%)	Support
Normal	99.16	99.27	99.21	7,500
Jammer	69.68	69.45	69.56	7,500
Spoofer	94.11	96.36	95.22	7,500
Interference	69.33	67.83	68.57	7,500
Overall	**83.07**	**83.23**	**83.14**	**30,000**

Table II reports per-class precision, recall, and F1-score on the held-out test set of 30,000 balanced samples. Normal signal classification achieves near-perfect F1 (99.21%) due to the distinct narrowband spectral characteristics of continuous-wave modulations. Spoofer classification achieves 95.22% F1, benefiting from the coherent phase structure of PSK modulations. Jammer (69.56% F1) and Interference (68.57% F1) exhibit lower scores, attributable to spectral overlap between wideband modulations assigned to different signal classes.

Table III tracks test accuracy at each stage of the quantization pipeline. BN folding via (4) and (5) is algebraically exact and introduces zero accuracy change within floating-point precision. INT8 symmetric quantization introduces 0.05 percentage-point degradation—well within the design target of less than 1% and consistent with the theoretical bound of $\pm0.5s_W$ per weight element [18]. Vivado synthesis targeting the Xilinx Zynq-7000 SoC available on the ZedBoard platform results in the power numbers presented in Table IV and resource utilization in Table V.

The on-chip power is shown in Table IV. The synthesized design satisfies all hold and pulse-width constraints, achieving a maximum operating frequency of 72.2 MHz. As shown in Table V, the design uses 18.82% of the total LUTs and 5.13% of the total flip-flops. This indicates that the proposed ResNet-1D inference datapath is a good fit for the target FPGA

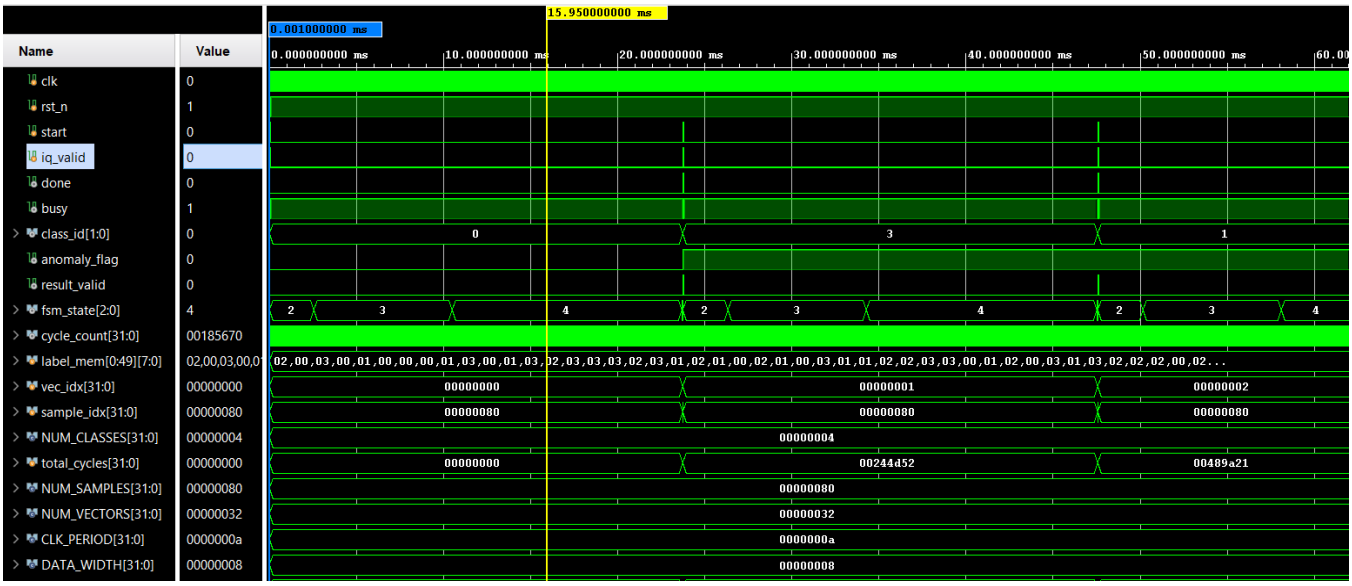


Fig. 3. Behavioral simulation showing incrementing cycle_count every inference, FSM state transitions, I/Q data streaming, and control signals.

TABLE III
TEST ACCURACY AT EACH STAGE OF THE QUANTIZATION PIPELINE

Pipeline Stage	**Accuracy (%)**	**Δ (%)**
FP32 Baseline	83.23	—
After BN Folding (FP32)	83.23	0.00
After INT8 Quantization	83.18	−0.05

TABLE IV
ON-CHIP POWER ESTIMATION ON XILINX XC7Z020CLG484-1 (ZYNQ-7000)

Parameter	**Value**	**Share (%)**
Total On-Chip Power	**0.671 W**	—
Dynamic Power	0.559 W	83
Device Static Power	0.113 W	17
Dynamic Power Breakdown		
Clocks	0.271 W	49
Signals	0.152 W	27
Logic	0.130 W	23
BRAM	0.005 W	1
I/O	<0.001 W	<1
Junction Temperature	32.7°C	—
Thermal Margin	52.3°C (4.4 W)	—

TABLE V
FPGA RESOURCE UTILIZATION ON XILINX XC7Z020CLG484-1 (ZYNQ-7000)

Resource	**Used**	**Available**	**Utilization (%)**
LUT	10,012	53,200	18.82
FF	5,461	106,400	5.13
BRAM	15	140	10.71
IO	1	200	0.50

logic fabric. The BRAM utilization of 10.71% indicates that both the weight ROM and intermediate feature maps can be comfortably stored in on-chip memory resources, which aligns with our 28.6 KB INT8 weight storage strategy. Input/Output (I/O) usage of 0.50% is also minimal due to the fully self-contained accelerator running from internal BRAM memory.

Fig. 3 shows the simulation waveform for the design, which, together with the behavioral simulation, indicates that the inference completes in 56 clock cycles. At the maximum post-implementation operating frequency of 72.2 MHz, this yields a realized hardware latency of 56×13.85 ns ≈ 0.78 μs. The measured cycle counter will increase from 0x35 to 0x39 when the LOAD state is executing, which is consistent with cycle-accurate designs.

The proposed accelerator is contextualized relative to other FPGA-based RF classification accelerators in Table VI. The proposed implementation achieves the lowest inference latency (0.78 μs) and power consumption (0.671 W) among comparable INT8 implementations by eliminating HLS design overhead and fitting the compact 29,252-parameter model entirely within on-chip BRAM. This results in an inference energy consumption of only 0.38 μJ, which is particularly attractive for battery-powered or thermally constrained edge deployments. Although binarized approaches such as FINN may achieve comparable or lower latency, they generally sacrifice representational capacity and classification accuracy. RFSoC-based deployments attain higher accuracy on the full 24-class modulation-recognition problem but rely on substantially larger and more power-hungry hardware platforms,

TABLE VI
COMPARISON WITH FPGA-BASED RF CLASSIFIERS

Metric	Proposed	DeepRadio [8]	hls4ml [12]	FINN-based [20]	RFSoC-AMC [9]
Model Architecture	ResNet-1D (8-layer)	Custom CNN	Configurable	BNN	VGG10/ResNet33
Dataset	RadioML 2018.01A	Proprietary RF	RadioML	RadioML	RadioML 2018.01A
Quantization	INT8 (PTQ)	Fixed-point 16-bit	Mixed fixed-point	1-bit (Binary)	INT8 (HLS)
HLS Dependency	**None (custom RTL)**	Partial	Full HLS	Full HLS	Full HLS
Signal Taxonomy	**4-class Signal**	Per-modulation	Per-modulation	Per-modulation	Per-modulation
Accuracy (%)	83.18	$\sim$80	$\sim$85	$\sim$80.1	$\sim$90
Latency (μs)	**0.78**	$\sim$5	$\sim$1–10	$\sim$283	$\sim$10
Power (W)	**0.671**	$\sim$2–5	$\sim$1–3	$\sim$3.86	$\sim$5–10
Energy/Inf. (μJ)	**0.38**	$\sim$10–25	$\sim$1–30	$\sim$1.21	$\sim$50–100
Weight Storage (KB)	**28.6**	$>$100	Variable	$\sim$187.5	$>$500
Target FPGA	Zynq-7000	Zynq UltraScale+	Various	Zynq-7000	RFSoC ZU+
Parameters	29,252	$>$100K	Variable	$>$1.5M	$>$1M

BNN: Binary Neural Network.

making direct latency and power comparisons less meaningful for edge-focused deployments.

V. CONCLUSION

This work has presented an end-to-end system design for real-time RF anomaly detection at the sensor edge. The utilization of ResNet-1D architecture resolved the problem of vanishing gradient associated with a standard 1D CNNs with the aid of identity and 1×1 projection skip connections for training a deeper eight-layer network comprising 29,252 weights on RadioML 2018.01A with 83.23% FP32 testing accuracy. The folded BN and INT8 quantized model only incurred a less than 0.05% accuracy degradation from the FP32 base model, thereby downsizing the required weight storage to 28.6 KB so that it can be fit in the on-chip BRAM of Zynq-7000 hardware platforms. Furthermore, our Vivado synthesis and implementation leads to a significantly small on-chip power consumption of 0.671 W, a minimal junction temperature of 32.7°C, and, consequently, a latency of 0.78 μs.

VI. ACKNOWLEDGMENTS

Portions of this manuscript were refined using AI-based writing assistants for grammar, clarity, and style. All scientific content, methodology, and analysis were developed and authored by the researchers.

REFERENCES

[1] A. C. S. Thakur and M. Imtiaz, "Deep learning for automatic modulation classification: A review," *Electronics*, vol. 15, no. 10, p. 2163, May 2026.

[2] H. Xu, Y. Liang, R. Xie, and Y. Kong, "The deep learning evolution in wireless physical layer communications: Applications, challenges, and evolutionary directions," *Sensors*, vol. 26, no. 11, p. 3609, Jun. 2026.

[3] DeepSig Inc., "RadioML 2018.01A dataset," 2018. [Online]. Available: https://www.deepsig.ai/datasets

[4] S. Ansari et al., "Attention-enhanced hybrid automatic modulation classification for advanced wireless communication systems: A deep learning-transformer framework," *IEEE Access*, vol. 13, 2025.

[5] F. Zhu et al., "Robust deep learning-based physical layer communications: Strategies and approaches," *IEEE Netw.*, 2025.

[6] S. Chatterjee, S. Ghosh, T. Ghosh, and H. Rahaman, "Architectural design and performance analysis of FPGA based AI accelerators: A comprehensive review," arXiv preprint arXiv:2603.08740, Feb. 2026.

[7] X. Chen, X. Tian, Q. Zheng, and W. Wang, "Attention mechanism enabled wireless modulation recognition network," *Eng. Lett.*, vol. 33, no. 4, Apr. 2025.

[8] E. Chang, D. Cheng, and F. T. Luk, "DeepRadio: Real-time FPGA deep learning for RF signal classification in electronic warfare," in *Proc. IEEE Mil. Commun. Conf. (MILCOM)*, Norfolk, VA, Nov. 2019, pp. 1–6.

[9] E. Chang *et al.*, "Real-time modulation classification on Xilinx RFSoC using deep neural networks," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, Seville, Spain, May 2020, pp. 1–5.

[10] D. J. M. Moss, D. Boland, P. Pourbeik, and P. H. W. Leong, "Real-time FPGA-based anomaly detection for radio frequency signals," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, Florence, Italy, May 2018, pp. 1–5.

[11] T. J. O'Shea, T. Roy, and T. C. Clancy, "Over-the-air deep learning based radio signal classification," *IEEE J. Sel. Topics Signal Process.*, vol. 12, no. 1, pp. 168–179, Feb. 2018.

[12] J. Duarte *et al.*, "Fast inference of deep neural networks in FPGAs for particle physics," *J. Instrum.*, vol. 13, no. 07, p. P07027, Jul. 2018.

[13] X. Wang, Y. Zhao, and Z. Huang, "A survey of deep transfer learning in automatic modulation classification," *IEEE Trans. Cogn. Commun. Netw.*, vol. 11, no. 3, pp. 1357-1381, June 2025.

[14] L. Zhang, T. Zhou, J. Yang, Y. Li, Z. Zhang, X. Hu, and Y. Peng, "A real-time signal measurement system using FPGA-based deep learning accelerators and microwave photonic," *Remote Sens.*, vol. 16, no. 23, p. 4358, Nov. 2024.

[15] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Las Vegas, NV, Jun. 2016, pp. 770–778.

[16] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.

[17] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. 32nd Int. Conf. Mach. Learn. (ICML)*, Lille, France, Jul. 2015, pp. 448–456.

[18] B. Jacob *et al.*, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE CVPR*, Salt Lake City, UT, Jun. 2018, pp. 2704–2713.

[19] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learn. Representations (ICLR)*, San Diego, CA, USA, May 2015.

[20] Y. Umuroglu et al., "FINN: A framework for fast, scalable binarized neural network inference," in *Proc. ACM/SIGDA Int. Symp. Field-Program. Gate Arrays (FPGA)*, Monterey, CA, Feb. 2017, pp. 65–74.